

Smart Energy Consumption & Grid Monitoring Analytics Platform

Project Documentation

Platform: Microsoft Azure

Prepared by: Anuj

1. Project Overview

This project builds a cloud-based analytics platform on Microsoft Azure that processes household energy consumption data in two ways at once: a batch pipeline that processes a real historical archive, and a streaming pipeline that simulates live smart-meter readings.

Both pipelines clean and enrich the data using the same logic, and both write into one shared database. That database then powers Power BI dashboards showing consumption patterns and power-quality issues, so the end result is a single, consistent view of energy usage regardless of whether the data arrived in a big historical batch or one reading at a time.

The goal was to practice a realistic, end-to-end Azure data engineering workflow: ingesting data, moving it through cloud storage, transforming it with Spark, loading it into a SQL database, and visualizing it — while also documenting the real problems encountered along the way.

2. Dataset Details

The project uses the “Individual Household Electric Power Consumption” dataset, a real, publicly available archive of one household's electricity usage recorded once per minute over roughly four years (about 2.07 million rows).

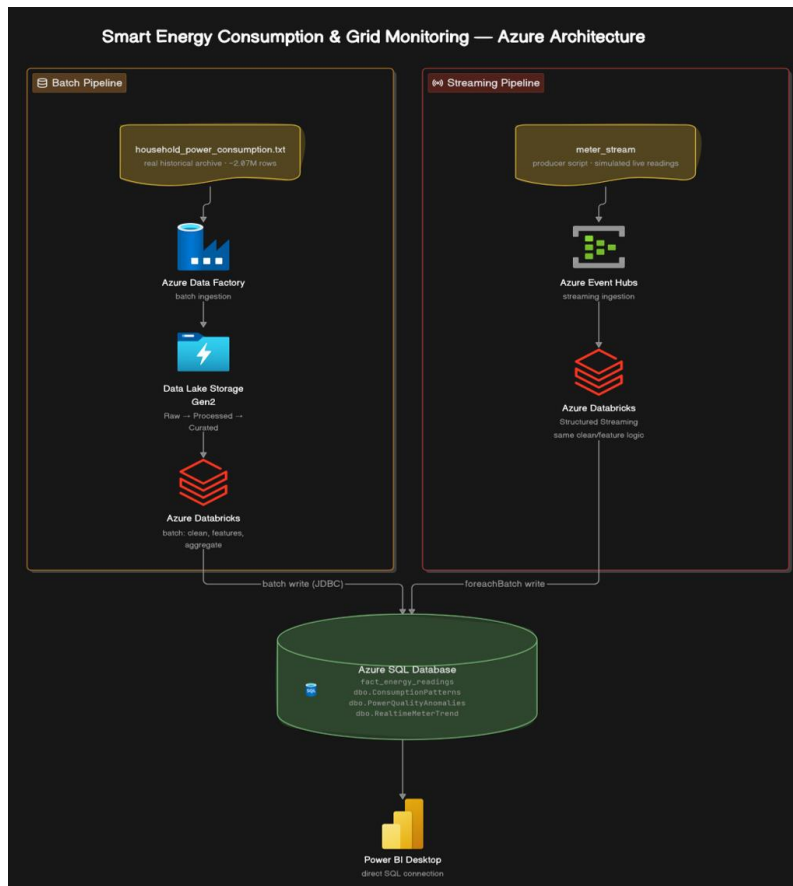
Each reading includes: date and time, global active and reactive power, voltage, current (global intensity), and three sub-metering values covering the kitchen, laundry, and water-heater/air-conditioning circuits. Since the dataset only covers one household, the project treats it as a single meter, referenced as HOUSE_001 throughout the database.

A known property of this dataset is that a small percentage of readings are missing and marked with a “?” character in the raw file. The pipeline treats these as genuine missing values rather than dropping the rows.

3. Architecture

The platform has two independent ingestion paths that both feed the same database, so the diagram below should be read as one system with two front doors rather than two separate systems.

Figure 1: Overall Azure Architecture



Batch path: the raw historical file is copied by Azure Data Factory into Data Lake Storage, then cleaned and transformed by an Azure Databricks notebook, then loaded into Azure SQL Database.

Streaming path: a small Python script simulates a live meter by sending readings to Azure Event Hubs at timed intervals. A Databricks Structured Streaming job consumes these in real time, applies the same cleaning logic as the batch path, and writes into the same database.

Both paths converge on Azure SQL Database, which is what Power BI connects to for the final dashboards.

4. Cloud Resources Used

The following Azure services were provisioned for this project, all within a single resource group so they could be managed and torn down together:

- Azure Data Factory — copies the historical file into cloud storage (batch ingestion)
- Azure Event Hubs — receives simulated live meter readings (streaming ingestion)

- Azure Data Lake Storage Gen2 — zoned storage with Raw, Processed, and Curated folders
- Azure Databricks — runs the PySpark batch and streaming transformation jobs
- Azure SQL Database — the shared database that both pipelines write into
- Power BI Desktop — dashboards connected directly to the database

Figure 2: Hierarchical Namespace Enabled on the Storage Account

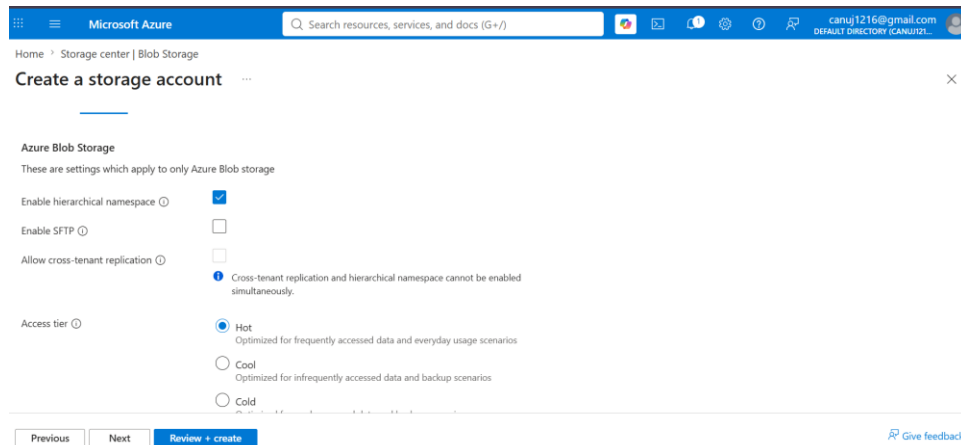


Figure 3: Raw, Processed, and Curated Storage Zones

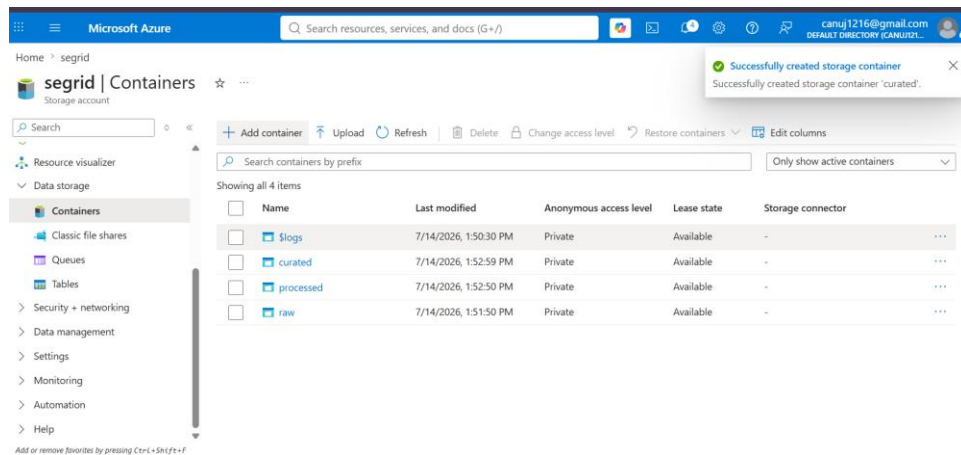
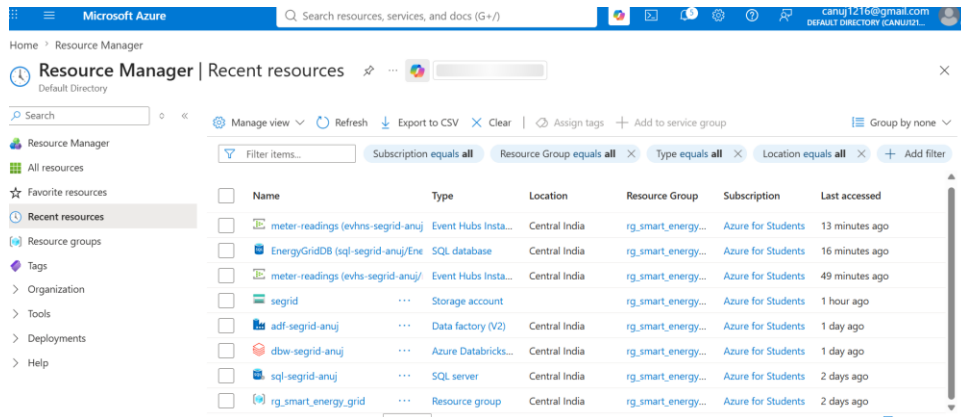


Figure 4: All Provisioned Azure Resources

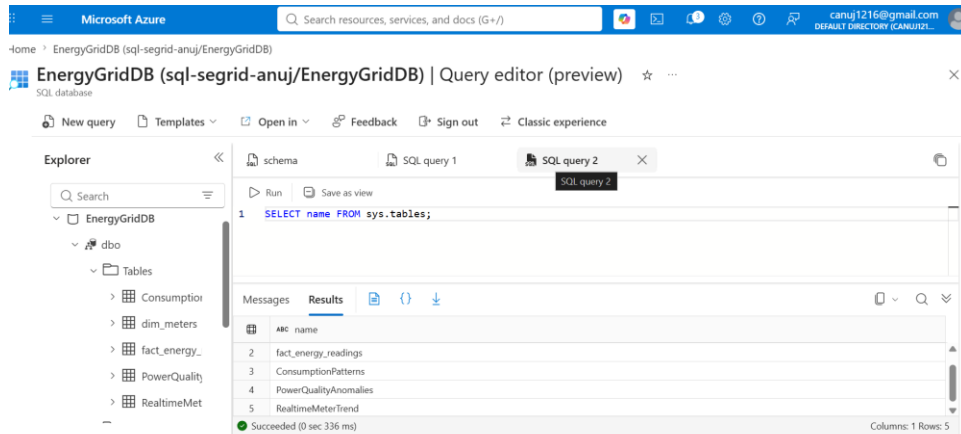


5. Database Design

The database holds five tables: one dimension table, one central fact table, and three tables built specifically to serve the project's three use cases.

- `dim_meters` — a small reference table describing each meter. Since the real dataset covers only one household, this table currently holds a single row (HOUSE_001), but keeps the design ready for more meters later.
- `fact_energy_readings` — the central table. Every reading from both the batch and streaming pipelines lands here, tagged with an `ingestion_source` column ('Batch' or 'Streaming') so the two can always be told apart.
- `dbo.ConsumptionPatterns` — average power usage by hour of day and weekday/weekend, used for Use Case 1.
- `dbo.PowerQualityAnomalies` — readings flagged for voltage outside the normal range, used for Use Case 2.
- `dbo.RealtimeMeterTrend` — designed to hold rolling averages from the streaming job for real-time monitoring (Use Case 3); see Section 8 for why this table has no data in the final submission.

Figure 5: All Five Tables Confirmed in EnergyGridDB



6. ETL Pipeline

6.1 Batch Pipeline

The batch pipeline runs in three stages, matching the three PySpark notebooks in the repository:

- Extraction (01_extract_raw): reads the raw file with an enforced schema, with no cleaning yet.
- Transformation (02_transform): converts the '?' missing-value marker into proper nulls, builds a real timestamp from the separate date and time columns, calculates a derived 'unmetered power' value, flags voltage anomalies, and removes duplicate readings.
- Loading (03_load): writes the cleaned data into fact_energy_readings and builds the two summary tables (ConsumptionPatterns and PowerQualityAnomalies).

Figure 6: The Three Batch Notebooks in Databricks

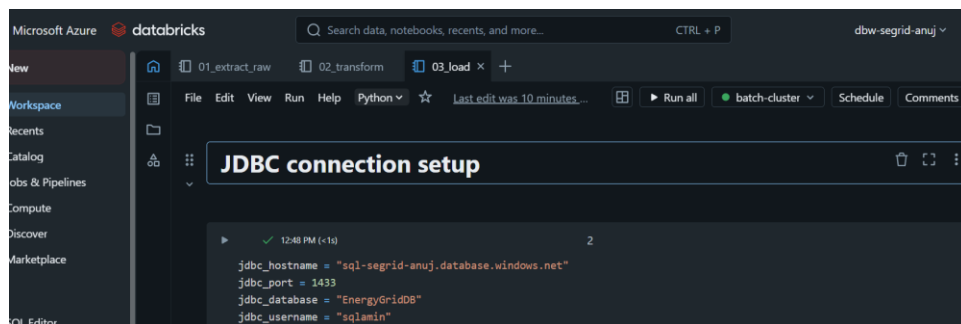


Figure 7: Azure Data Factory Batch Pipeline Run

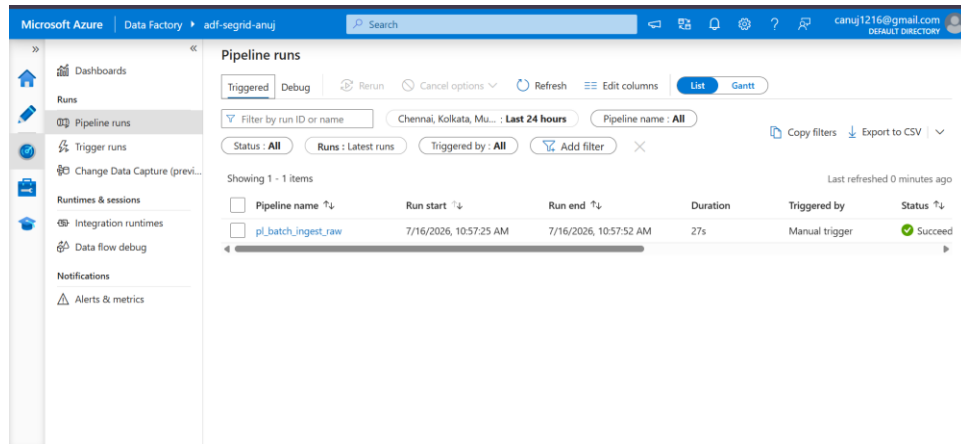
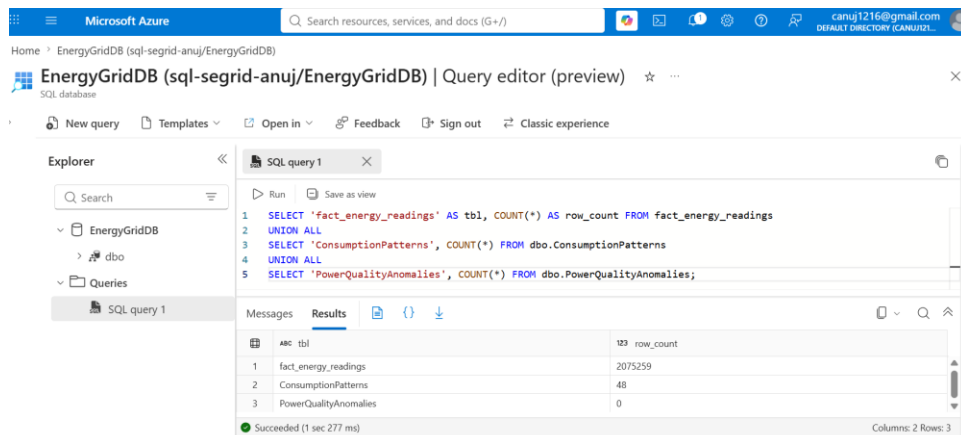


Figure 8: Row Counts Confirming the Batch Load



6.2 Streaming Pipeline

Since there is no real live meter for this project, a small Python script (producer_script.py) simulates one. It replays readings from the same historical file at a steady pace and sends each one to Azure Event Hubs, occasionally injecting an out-of-range voltage value so the anomaly detection has something real to catch.

A Databricks Structured Streaming notebook (stream_transform) continuously reads from Event Hubs, applies the same cleaning and anomaly-detection logic as the batch pipeline, and writes each micro-batch into the same database tables.

Figure 9: Azure Event Hubs Metrics — Messages Arriving from the Producer Script

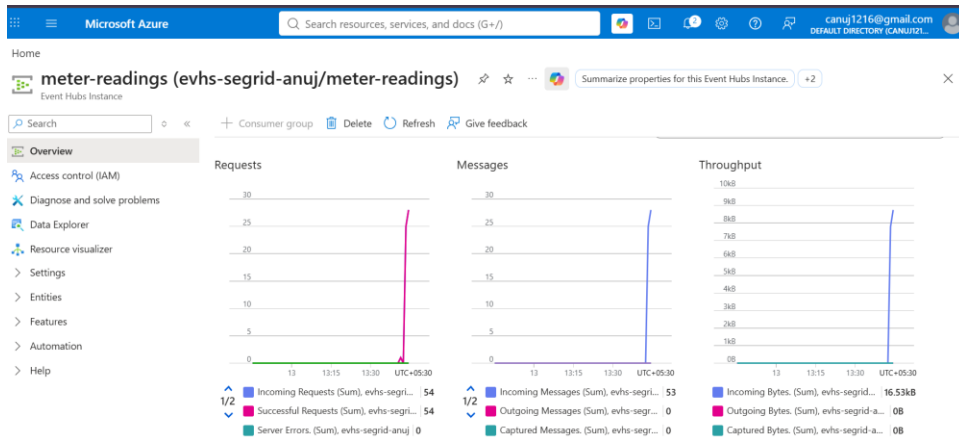


Figure 10: The Structured Streaming Job Actively Running

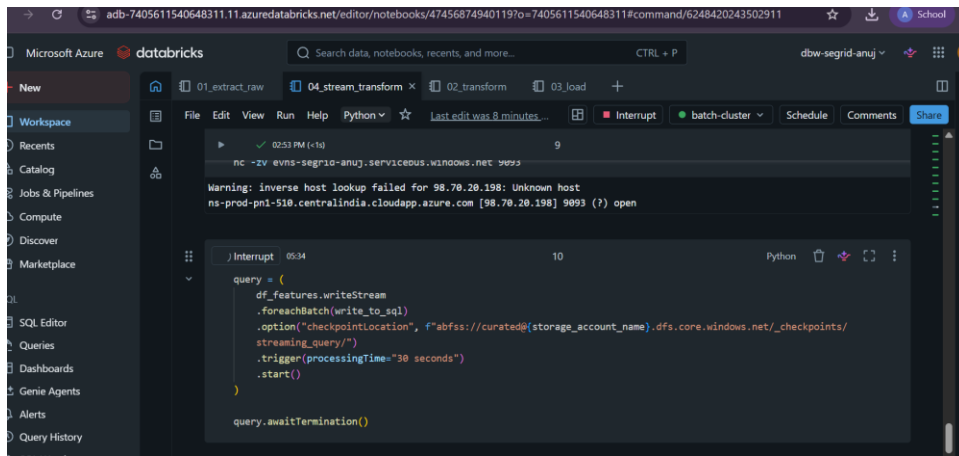


Figure 11: The Streaming Query Shown as an Active Job in the Spark UI

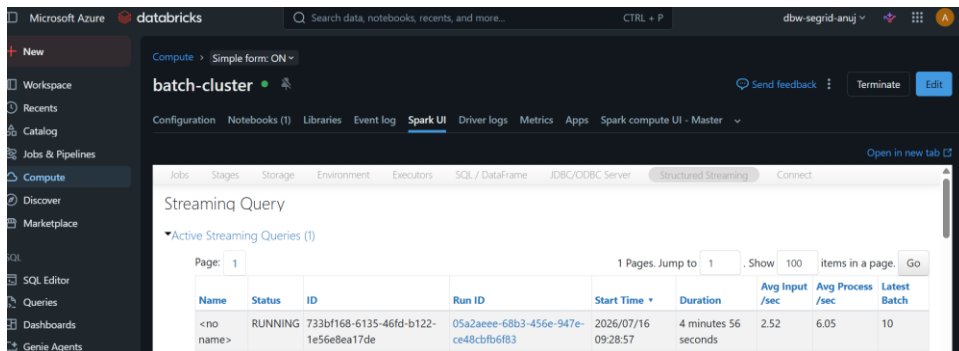


Figure 12: fact_energy_readings Holding Rows from Both Pipelines

Microsoft Azure | Search resources, services, and docs (G+/)

Home > EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

SQL database

Search

Explorer

- EnergyGridDB
 - dbo
 - Queries
 - SQL query 1
 - SQL query 2

SQL query 2

```

1 SELECT ingestion_source, COUNT(*) AS row_count
2 FROM fact_energy_readings
3 GROUP BY ingestion_source;

```

Messages Results

Msg	Msg	Msg
1	Batch	2075259
2	Streaming	482

Succeeded (5 sec 595 ms) Columns: 2 Rows: 2

7. SQL Analysis

Ten SQL queries were written against the curated database to answer practical questions about the data — covering grouping, filtering, views, and window functions. Eight are shown below as evidence; the full set of ten scripts is saved in the repository under `sql/analysis/analysis_queries.sql`.

Task 1: Average Power Usage by Hour of Day (Weekday vs. Weekend)

Microsoft Azure | Search resources, services, and docs (G+/)

Home > EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

SQL database

Search

Explorer

- EnergyGridDB
 - dbo
 - Queries
 - Task 1

Task 1

```

1 SELECT
2 CASE WHEN is_weekend = 1 THEN 'Weekend' ELSE 'Weekday' END AS day_type,
3 DATEPART(HOUR, reading_timestamp) AS hour_of_day,
4 AVG(global_active_power_kw) AS avg_global_active_power_kw
5 FROM fact_enerev_readings

```

Messages Results

Msg	Msg	Msg	Msg
1	Weekday	0	0.568563
2	Weekday	1	0.467770
3	Weekday	2	0.435255
4	Weekday	3	0.411625

Succeeded (6 sec 67 ms) Columns: 3 Rows: 48

Task 2: Top 10 Voltage Anomalies by Deviation

Microsoft Azure | Search resources, services, and docs (G+)

Home > EnergyGridDB (sql-segrid-anuj/EnergyGridDB)

EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

SQL database

New query | Templates | Open in | Feedback | Sign out | Classic experience

Explorer

Search

EnergyGridDB

dbo

Queries

Task 1

Task 2

Task 1

Run | Save as view

```
1 SELECT TOP 10
2   reading_timestamp,
3   anomaly_type,
4   voltage_v,
5   severity,

```

Messages | Results

	reading_timestamp	anomaly_type	voltage_v	severity	voltage_deviation
1	2010-10-051222:03:00.000000...	Voltage Anomaly	188.62	Medium	49.379999999999995
2	2010-10-05119:17:00.000000...	Voltage Anomaly	189.11	Medium	48.8899999999999986
3	2010-10-05119:33:00.000000...	Voltage Anomaly	190.89	Medium	47.1100000000000014
4	2010-10-05122:49:00.000000...	Voltage Anomaly	192.66	Medium	45.34

Succeeded (0 sec 551 ms)

Columns: 5 Rows: 10

Task 3: Saved View — Total Daily Energy Consumption

Microsoft Azure | Search resources, services, and docs (G+)

Home > EnergyGridDB (sql-segrid-anuj/EnergyGridDB)

EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

SQL database

New query | Templates | Open in | Feedback | Sign out | Classic experience

Explorer

Search

EnergyGridDB

dbo

Queries

Task 1

Task 2

Task 3

Task 1

Run | Save as view

```
1 CREATE OR ALTER VIEW dbo.vw_DailyEnergyConsumption AS
2 SELECT
3   CAST(reading_timestamp AS DATE) AS reading_date,
4   SUM(global_active_power_kw / 60.0) AS total_kwh -- readings are per-minute; kW/60 = kWh per reading
5 FROM fact_enerenv_readlines
```

Messages | Results

	reading_date	total_kwh
1	2006-12-16	20.1529201
2	2006-12-17	56.5076181
3	2006-12-18	36.7303860
4	2006-12-19	27.7698528

Succeeded (0 sec 793 ms)

Columns: 2 Rows: 1433

Task 4: Hours Ranked by Average Power Draw

Microsoft Azure | Search resources, services, and docs (G+)

Home > EnergyGridDB (sql-segrid-anuj/EnergyGridDB)

EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

SQL database

New query | Templates | Open in | Feedback | Sign out | Classic experience

Explorer

Search

EnergyGridDB

dbo

Queries

Task 1

Task 2

Task 3

Task 4

Task 1

Run | Save as view

```
1 SELECT
2   hour_of_day,
3   avg_power,
4   RANK() OVER (ORDER BY avg_power DESC) AS power_rank
5 FROM (
6   SELECT
```

Messages | Results

	hour_of_day	avg_power	power_rank
1	20	1.899593	1
2	21	1.877824	2
3	19	1.733285	3

Succeeded (0 sec 553 ms)

Columns: 3 Rows: 24

Task 6: Running Daily Total of Readings by Ingestion Source

Microsoft Azure

EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

Explorer

Task 6

```

1 SELECT
2   reading_date,
3   ingestion_source,
4   daily_count,
5   SUM(daily_count) OVER (
6     PARTITION BY ingestion_source ORDER BY reading_date

```

Messages Results

	reading_date	ABC ingestion_source	123 daily_count	123 running_total
1	2006-12-16	Batch	396	396
2	2006-12-17	Batch	1440	1836
3	2006-12-18	Batch	1440	3276

Succeeded (0 sec 526 ms)

Task 7: Days with Unusually High Average Power Draw

Microsoft Azure

EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

Explorer

Task 7

```

1 SELECT
2   d.reading_date,
3   d.avg_daily_power
4 FROM (
5   SELECT
6     PARTITION BY ingestion_source ORDER BY reading_date

```

Messages Results

	reading_date	e* avg_daily_power
1	2006-12-23	3.314851
2	2006-12-16	3.053474
3	2007-02-03	2.798418

Succeeded (0 sec 757 ms)

Task 8: Share of Power Usage by Sub-Metered Circuit vs. Unmetered Load

Microsoft Azure

EnergyGridDB (sql-segrid-anuj/EnergyGridDB) | Query editor (preview)

Explorer

Task 8

```

9 AS pct_kitchen,
10 SUM(sub_metering_laundry_wh) * 100.0 / NULLIF(
11 SUM(sub_metering_kitchen_wh) + SUM(sub_metering_laundry_wh) +
12 SUM(sub_metering_waterheater_ac_wh) + SUM(unmetered_power_wh), 0

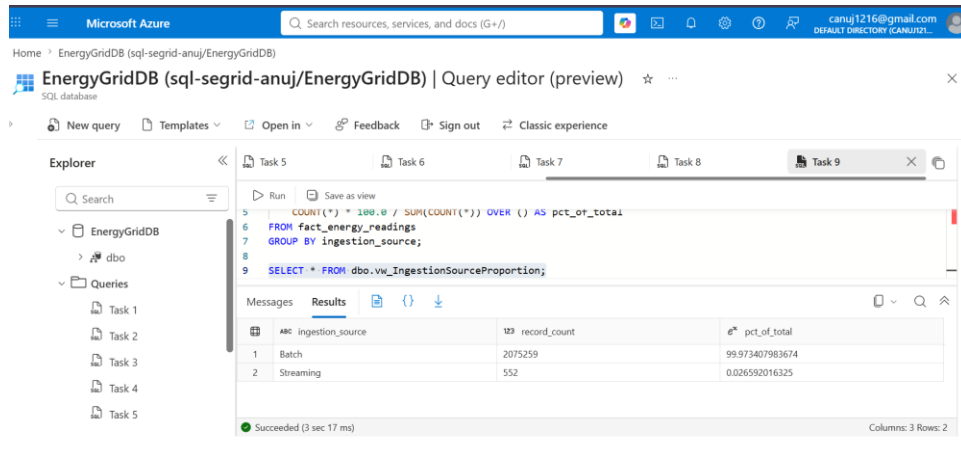
```

Messages Results

	e* total_kitch...	e* total_laund...	e* total_water...	e* total_unme...	e* pct_kitchen	e* pct_laundry	e* pct_waterhe...	e* pct_unmet...
1	2299135.00	2663134.00	13237096.00	19097774.42	6.164373	7.140317	35.490914	51.204394

Succeeded (0 sec 514 ms)

Task 9: Saved View — Proportion of Readings by Ingestion Source



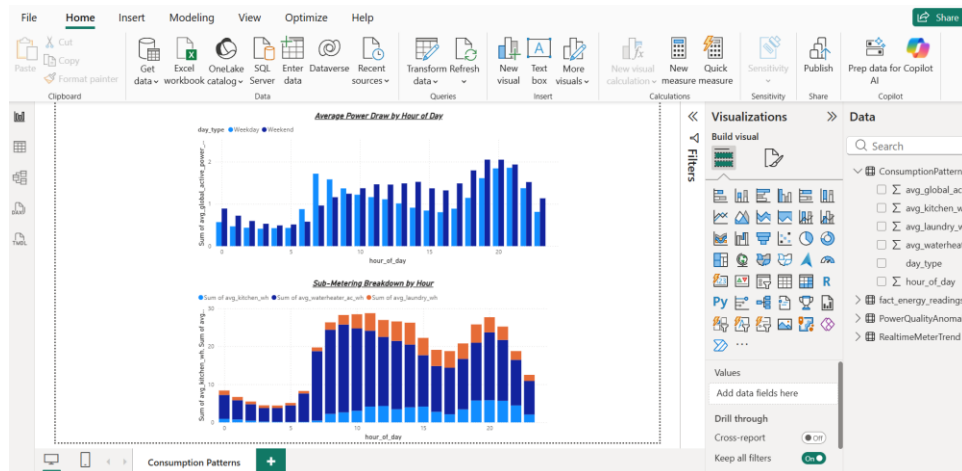
8. Visualizations

Two Power BI dashboard pages were built, connected directly to Azure SQL Database using Import mode, which means the dashboards keep working even after the underlying Azure resources are removed.

8.1 Consumption Patterns

This page answers Use Case 1: when does this household use the most power, and which appliances drive that usage? It shows average power draw by hour of day (split by weekday/weekend), and a breakdown of that usage across the kitchen, laundry, and water-heater/AC circuits.

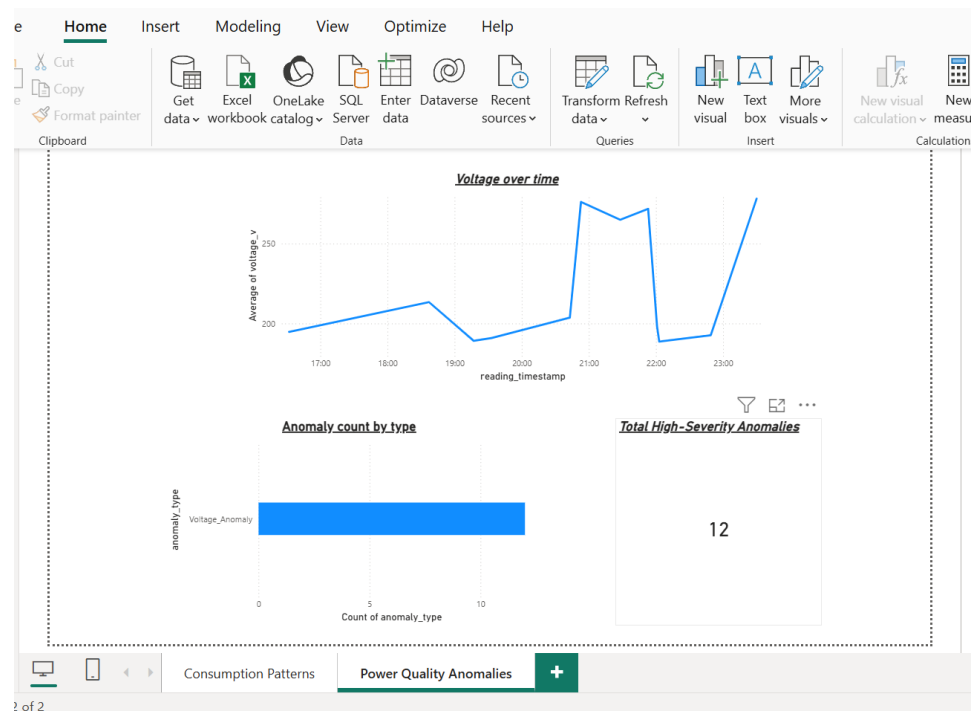
Figure 13: Consumption Patterns Dashboard Page



8.2 Power Quality Anomalies

This page answers Use Case 2: are there moments where the electrical supply itself looks unhealthy? It shows voltage over time, a count of anomalies by type, and a total count of high-severity anomalies.

Figure 14: Power Quality Anomalies Dashboard Page



8.3 Real-Time Monitoring (Use Case 3)

A third page was planned to show live rolling averages from the streaming pipeline. It was not included in the final Power BI file because its underlying table, `dbo.RealtimeMeterTrend`, has no data — the Event Hubs resource was intentionally deleted after the streaming pipeline was tested and confirmed working, in order to avoid ongoing charges on a student subscription. The underlying capability was verified directly instead: Figures 9–12 show the streaming job actually running and successfully writing data. See Section 9 for more on this decision.

9. Challenges

9.1 Streaming with Databricks and Kafka

Getting the streaming pipeline working was the hardest part of the project. Azure Event Hubs can be read using the Kafka protocol, which is how Databricks Structured Streaming connects to it, but this was new territory for me. It took several rounds of troubleshooting to get right — including discovering that Event Hubs' Basic pricing tier

does not support the Kafka protocol at all, which meant creating a new namespace on the Standard tier partway through. Along the way, a related Spark API change caused a separate error that took me some deep digging to trace back to its actual cause.

9.2 Database Connection and Firewall Errors

Azure SQL Database only accepts connections from IP addresses explicitly allowed on its firewall. Since a laptop's IP address can change between sessions (especially on different networks), I ran into “your IP isn't allowed to access this server” errors more than once, and had to re-add the current IP address each time. This was a useful reminder that cloud security defaults are strict by design, and that connectivity issues are often a configuration step rather than a real fault.

9.3 Incomplete Real-Time Dashboard

The third Power BI page (real-time monitoring) was not included in the final submission. The underlying streaming logic was built and proven to work — the streaming job ran successfully and wrote real rows into the database, as shown in Section 6.2 — but the Event Hubs resource was deleted afterward to avoid ongoing cost on a student Azure subscription, which left the table empty. This was a deliberate cost-versus-completeness tradeoff rather than an oversight.

10. Learnings

- Building the same cleaning logic twice (once for batch, once for streaming) makes it much easier to trust that both pipelines will produce consistent results — but it also means any fix has to be applied in two places.
- Serverless/free-tier Azure resources (like the SQL Database used here) auto-pause when idle, which can look like a real failure the first time a connection is attempted after a break. Knowing to simply retry after a short wait saved a lot of confusion.
- Cloud costs are easy to control by matching each resource's tier to what the project actually needs (for example, Basic Event Hubs was enough until Kafka support was needed), and by deleting resources as soon as they're no longer needed rather than at the very end.
- Reading the full error message — not just the first line — mattered more than expected. Several errors in this project only made sense once the underlying “Caused by” section was found further down the stack trace.

11. Conclusion

This project delivered a working, end-to-end energy analytics platform on Azure: a real historical archive processed through a batch pipeline, a simulated live meter processed through a streaming pipeline, both converging on a shared database, and two Power BI dashboards built directly on top of it.

Not everything originally planned made it into the final submission — the real-time dashboard page was consciously scoped out to prioritize a complete, working core pipeline over a partially-finished larger scope. The challenges faced along the way, particularly around streaming and cloud networking, reflect genuine hands-on experience with real Azure services rather than a purely theoretical exercise.